Ders 9 Genişletilmiş Notu: Kod Kusurları ve Refactoring Eylemleri: Duplicated Code, Long Method, Long Parameter List ve Large Class

Ders 9 Genişletilmiş Notu: Kod Kusurları ve Refactoring Eylemleri: Duplicated Code, Long Method, Long Parameter List ve Large Class

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Refactoring ve Kod Kusuru Tanımlarının Tekrarı
- 2. Duplicated Code (Yinelenen Kod)
- 3. Extract Method (Metot Çıkartma)
- 4. Long Method (Aşırı Uzun Kod)
- 5. Pull Up Method ve Template Method ile Yinelenen Kodu Toplamak
- 6. Decompose Conditional (Karar Verme Komutunu Parçalama)
- 7. Long Parameter List (Uzun Parametre Listesi)
- 8. Introduce Parameter Object (Parametre Nesnesi Oluşturma)
- 9. Preserve Whole Object (Tüm Nesneyi Gönder)
- 10. Replace Temp with Query (Geçici Değişken Yerine Erişim Metodu)
- 11. Replace Method with Method Object (Metot Yerine Yeni Türden Nesne)
- 12. Large Class (Aşırı Büyük Sınıf) ve Metrikler
- 13. Extract Class (Sınıf Çıkartma)
- 14. Extract Subclass (Alt Sınıf Çıkartma)
- 15. Duplicate Observed Data (Gözlenen Verinin Çoğullanması)
- Örnek Sorular
- Dersin Toparlaması

1. Refactoring ve Kod Kusuru Tanımlarının Tekrarı

Geçen hafta tasarım kalıpları bitirilmiş, kod kusurları (code smells) ve refactoring konusuna başlanmıştı. Refactoring'in ve kod kusurunun ne olduğu, refactoring'in birim sınamalarıyla birlikte yapıldığı konuşulmuş, birim sınamaları için JUnit 3 tanıtılmış ve örnekler yapılmıştı. JUnit 4'ün getirdiği annotation kullanımı ve istisna sınaması da anılmıştı; ders kapsamındaki örnekler için 3. sürüm yeterlidir. Bu ders kod kusurları ve onları düzelten refactoring eylemleriyle devam etti.

1.1 Tanımlar

Refactoring, bir kodun dış arayüzünü değiştirmeden iç çalışma yapısını değiştirmektir. Amacı daha iyi iç kalite özelliklerine ulaşmak ve genelde gelen gereksinimlere karşı daha esnek çözümler sunmaktır.

Kod kusuru "hatalı kod" diye algılanmamalıdır. Kodda bir bug yoktur; program doğru çalışır, ihtiyacı karşılar. Ama kodlamada, kullanılan dilin ya da yaklaşımın önerilerine aykırı yapılmış

şeyler, ya da tasarım kusurları vardır. Bunlar yakında gelecek değişikliklerde sorun çıkaracak kodlardır.

1.2 Bu bölümün kaynağı ve kapsamı

İçerik Martin Fowler'ın "Refactoring" kitabından düzenlenmiştir. Kitaptaki bütün kod kusurları işlenmiyor; hoca bir seçme yaptı. Seçmenin mantığı şudur: bir kod kusurunu düzeltmek için birden fazla refactoring eylemi kullanılabilir ve bir refactoring eylemi birden fazla kusuru düzeltebilir. Yani kod kusurları ile refactoring eylemleri arasında birebir değil, M×N bir ilişki vardır. Hoca bu ilişkiler içinde en çok iç içe geçmiş olanları seçerek ders notlarına aldı. Daha fazlasını isteyenler Fowler'ın kitabına bakabilir.

Kod kusurları nispeten küçük ölçekli kusurlardır. Daha büyük ölçekli kusurlar "anti-patterns" konusunun içindedir; isteyenler bunu ayrıca araştırabilir.

> **Hocanın vurgusu:** Kod kusurlarının ve refactoring eylemlerinin İngilizce özgün adları akılda kalmalıdır. Hoca bu adların anlamlarını çok iyi anlattığını da ekledi. Bazı adlar (örneğin ileride görülecek "Shotgun Surgery") Türkçeye tam oturmadığı için olduğu gibi kullanılacak.

1.3 Önce ve sonra gösterimi, üye alan adlandırması

Kod örnekleri, diyet reklamlarındaki "öncesi ve sonrası" fotoğrafları gibi yan yana gösterilir: solda kodun önceki hali, sağda refactor edilmiş hali.

Slayttaki "Üye alan adlandırma" notu (ekrandan, 00:17:50): kod örneklerinde alt çizgiyle başlayan değişkenler nesnenin üye alanlarıdır; IDE'lerde ise üye alanlar farklı renkle vurgulanır. Hoca bunun C++ geleneğini yaşatmak için değil, siyah beyaz malzemede IDE renklendirmesinden yararlanılamadığı yerlerde neyin ne olduğunu ayırt etmek için yapıldığını söyledi.

2. Duplicated Code (Yinelenen Kod)

İlk kod kusuru Duplicated Code'dur (ekrandan, 00:17:50). Aynı kodun (birebir, kelimesi kelimesine) ya da çok ufak farklılıklarla çok benzer bir kod yapısının birden fazla yerde tekrarlanmasıdır.

Bu kodun işlevselliğinde bir değişiklik gerektiğinde (bulunan bir hata da olsa, gereksinimdeki bir değişiklik de olsa) kodun tekrarlandığı her yerde aynı işlemleri tekrarlamak gerekir. Ortaya çıkan sonuç bir bakım kabusudur.

Bu kusuru düzeltmek için ilk akla gelen refactoring eylemi Extract Method'dur: yinelenen kodu bir metot altında toplamak. Yinelenen kodu toplamaya yönelik başka eylemler de vardır (Pull Up Method, Template Method kalıbı); bunlara aşağıda değinilecek.

3. Extract Method (Metot Çıkartma)

3.1 Tanım

Slayttaki tanım (ekrandan, 00:20:00):

- Bir metot içindeki kodun bir kısmını yeni bir metot olarak belirlemeye dayanır.
- Bu yapılandırma birçok kod kusurunu (smell) düzeltir.
- Oluşturulan yeni metoda amacını açıkça belirten, isabetli bir ad verilmelidir. Böylece alt düzey ayrıntılarla uğraşmadan kodun anlaşılabilirliği artar.

3.2 Birinci örnek: printOwing

```
/*** ÖNCE: ***/
void printOwing(double amount) {
   printBanner();
   //print details
   System.out.println("name:" + _name);
   System.out.println("amount" + amount);
```

```
}

/*** SONRA: ***/
void printOwing(double amount) {
   printBanner();
   printDetails(amount);
}
void printDetails (double amount) {
   System.out.println("name:" + _name);
   System.out.println("amount" + amount);
}
```

(ekrandan, 00:20:00)

`printOwing` borç miktarını parametre olarak alır. Önce bir başlık (banner) yazılır, sonra bir yorumla belirtilen ayrıntılar yazılır: kişinin adı ve borç miktarı. `_name` bir üye alandır, `amount` ise gelen parametredir. Refactoring'de ayrıntıları yazan satırlar `printDetails` metoduna çıkarılır ve eski metot içinden bu yeni metot çağrılır.

3.3 Neden yaradı? Anlaşılabilirlik

`printOwing` önemli oranda kısaldı. Hoca bunu insanın bilgi işleme sınırıyla açıkladı: bir insanın bir bakışta işleyip üzerinde düşünebileceği, dikkatini odaklayabileceği bilgi parçası sayısı "5 artı eksi 2"dir. Bu, kişilerin içsel yeteneklerine bağlıdır: daha az yetenekli olanlar aynı anda üç bileşenle, daha yetenekli olanlar yedi bileşenle çalışabilir. Kod incelemek de böyledir: büyük, uzun bir kodu anlamak zordur. Ama kod ana hatlarıyla alt işler şeklinde toparlanır ve sonra hangi alt işin daha ayrıntılı anlaşılması gerekiyorsa ona odaklanılırsa anlamak kolaylaşır.

> **Ders sonrası not:** Bu konuda en çok bilinen çalışma George Miller'ın "7 artı eksi 2" bulgusudur. Hocanın verdiği "5 artı eksi 2" (3 ile 7 arası) büyük olasılıkla bu sınırı daha düşük bulan sonraki çalışmalara dayanıyor; altyazıda rakamın Whisper hatası olma olasılığı da vardır.

Yeni haliyle `printOwing`'in ana hatları açıkça görünür: bir banner yazılıyor, bir de ayrıntılar yazılıyor. Ayrıntılarla ilgili bir sorun varsa `printDetails` metoduna gidilir ve daha odaklı incelenir; banner'da sorun varsa `printBanner`'a bakılır.

3.4 Yorum satırı yerine anlamlı ad

Dikkat edilirse sonraki halde hiç yorum yoktur. Programlamaya başlarken "yaptığınız işleri yorum satırlarıyla anlatın" diye öğretilir, çünkü insan kendi yazdığı kodu bile altı ay sonra anlamakta zorlanır. Ciddi yazılım projelerinde ekip çalışması vardır: bir kişinin yazdığı kodu başka biri sınar, başka biri bakımını yapar. Bu yüzden anlaşılabilirlik çok önemlidir. Buradaki yaklaşım, anlaşılabilirliği kodun içine o kadar yedirmektir ki yorum satırlarına bile gerek kalmasın. Ayrıntıların yazıldığını metodun adından anlarız. Anlaşılabilirlik yükseldikçe bakım kolaylığı da iyileşir.

3.5 İkinci örnek: üç işe bölünen printOwing

Daha uzun bir `printOwing` metodu (ekrandan, 00:30:00):

```
/*** ÖNCE: ***/
void printOwing( ) {
  Enumeration e = _orders.elements();
  double outstanding = 0.0;

  // print banner
  System.out.println("*****************");
  System.out.println("* Customer Owes *");
  System.out.println("*****************");

  // calculate outstanding
```

```
  while (e.hasMoreElements()) {
    Order each = (Order) e.nextElement();
    outstanding += each.getAmount();
  }

  //print details
  System.out.println ("name:" + _name);
  System.out.println ("amount" + outstanding);
}
```

`_orders` bir üye alandır. Siparişlerin içinde dolaşmak için bir `Enumeration` alınır, kümülatif borç toplamı sıfırlanır. Önce banner yazdırılır, sonra döngüde her siparişin bedeli toplam borca eklenir, en sonda ayrıntılar (isim ve borç) yazılır. Metotta üç yorum, yani üç ayrı iş vardır.

```
/*** SONRA: ***/
void printOwing( ) {
  printBanner();
  printDetails(getOutstanding());
}

void printBanner() {
  System.out.println("*****************");
  System.out.println("* Customer Owes *");
  System.out.println("*****************");
}

double getOutstanding() {
  double outstanding = 0.0;
  for (Order each : _orders)
    outstanding += each.getAmount();
  /* for-each ile döngü kısaldı! */
  return outstanding;
}

void printDetails (double outstanding) {
  System.out.println ("name:" + _name);
  System.out.println ("amount" + outstanding);
}
```

Her iş kendi metoduna çıkarılır. Refactoring'in özü gereği `printOwing` bozulmaz, yerinde kalır, çünkü refactoring'in amacı dışarıya verilen hizmet arayüzünü değiştirmeden iç çalışmayı değiştirmektir.

> **Hocanın vurgusu:** Yaptığımız refactoring eylemleri bizi unit test kodlarımızı değiştirmek zorunda bırakmamalıdır.

Ayrıntılar yazılırken kümülatif toplam gerektiğinden ve toplam alma işi `getOutstanding` adlı ayrı metoda çıkarıldığından, `printDetails` çalışmadan önce `getOutstanding` hesaplanır ve sonuç `printDetails`'e verilir. Toplam hesaplanırken `while` döngüsündeki iş Java'nın for-each döngüsüyle yapıldı; üç satırlık döngü iki satıra indi. Slayttaki yorum yalnız bunu söylemeyi unutmamak için yazılmıştır; normalde yorum yazılmayacak.

3.6 Görünürlük

`printOwing` public ise public kalır. Çıkarılan diğer üç metodun public olması gerekmez; tasarımın geri kalanına ve ihtiyaçlara göre karar verilir. Başka yerden çağrılmaları gerekmiyorsa private bile yapılabilirler.

3.7 Kazanımlar ve olası dezavantajlar

Extract Method sayesinde:

- Uygulama mantığının bazı kısımlarının program içinde paylaşımı kolaylaşır.

- Anlaşılabilirlik artar.
- Yeniden kullanım olasılığı artar: odaklanmış bir işi yapan kod, aynı işe ihtiyaç duyan başka projelerde de rahatlıkla kullanılabilir. Uyumu düşük, aynı anda üç dört farklı iş yapan bir kod parçası ise olduğu gibi yeniden kullanılamaz, mutlaka düzenlemek gerekir. Kod odaklanmış parçalara ayrıldığında yalnız gereken parça seçilebilir.
- Uyum yükselir.
- Adı ile kendini açıklayan metotlar yazılır.

Hoca iki olası itirazı da tartıştı. Birincisi, böl ve yönet mantığıyla büyük kod küçük parçalara bölününce yönetilecek parça sayısı artar. Bu bir dezavantajdır, ama anlaşılabilirlik ve uyumun yükselmesi gibi ciddi kazanımlar karşısında biraz daha fazla metot olması sorun değildir; "günün sonunda kârdayız".

İkincisi, bir metotla yapılan iş birkaç metoda bölünüp metot içinde metot çağrıları ortaya çıktıkça çağırma ek yükü artar. Hoca bunu "context switching overhead" diye andı. Haklı bir itirazdır, ama günümüzde (hatta 10-20 yıldır) çok önemli değildir: hem C++ gibi derleyiciler hem Java Sanal Makinesi ya da .NET Runtime bu ek yükü önemli oranda azaltacak şekilde güncellenmiştir. Ayrıca sistemde çalışma anı başarımı açısından bir darboğaz varsa büyük olasılıkla başka bir yerdedir; buradan gelecek bir iki milisaniyelik ek yük hissedilmez bile.

> **Ders sonrası not:** Hoca "context switching" ve "1-2 milisaniye" dedi. Burada kastedilen metot (fonksiyon) çağırma ek yüküdür; "context switch" terimi genelde işletim sisteminin süreç ya da iş parçacığı değiştirmesi için kullanılır. Basit bir metot çağrısının yükü günümüz donanımında büyük olasılıkla nanosaniye düzeyindedir ve derleyiciler ile JIT derleyicileri satır içi açma (inlining) ile bunu çoğu zaman tamamen kaldırır. Hocanın asıl sonucu (ek yükün ihmal edilebilir olduğu) bununla uyumludur.

4. Long Method (Aşırı Uzun Kod)

İkinci kod kusuru Long Method'dur (ekrandan, 00:25:20). Slayttaki maddeler:

- Bir metodun uzunluğu arttıkça:
  - Anlaşılması zorlaşır,
  - Değişikliklerden (hata düzeltmeleri ya da gereksinim değişiklikleri) etkilenme olasılığı artar,
  - Birden fazla ilgi alanını kapsama olasılığı artar (uyumun düşmesi).
- Çözüme götürecek düzenlemeler:
  - Extract Method: uyumlu (belli bir ortak amaca yönelik) komutlar seçilerek bir metot altında toplanır.
  - Aşırı uzun kodu kısaltmaya yönelik diğer düzenlemelere ileride değinilecek.

Hoca uyum konusunu biraz açtı: kod ne kadar uzunsa orada o kadar çok iş yapılıyor demektir; bu işlerin birbirinden uzak, farklı alanlara yönelik olma olasılığı da artar. Bu yüzden odaklanmış koda ulaşılmaya çalışılır.

4.1 Sayısal bir örnek

Yirmi satırlık tek bir metot yerine diyelim sekiz satırlık üç metot yapılsın. $3 \times 8 = 24$; kodun geneli 20 satırdan 24 satıra çıktı. Ama eskiden bir sorunu anlamak için monolitik 20 satırı anlamak gerekirken, şimdi duruma göre üç metottan yalnız birine, yani 8 satıra odaklanmak yeterlidir. Üstelik ayrı metotlardaki üç iş kendi içinde daha uyumludur.

Böylece Extract Method hem Duplicated Code hem Long Method kusurlarını düzeltmekte kullanılabilen bir refactoring eylemidir. Uzun kodu kısaltmak için başka eylemler de var: Decompose Conditional, Replace Temp with Query ve Replace Method with Method Object (bölüm 6, 10 ve 11).

5. Pull Up Method ve Template Method ile Yinelenen Kodu Toplamak

5.1 Pull Up Method

Duplicated Code için kullanılabilecek bir başka eylem Pull Up Method'dur (metodu yukarı çekme). Yinelenen kod parçası bir kalıtım hiyerarşisindeki sınıflarda yer alıyorsa, o parçayı

kalıtım ağacında üst sınıfa çekmek en doğrusudur. Kardeş ya da kuzen sınıflarda kopyalanan kod hiyerarşide yukarı çekilirse tekrarlanmasına gerek kalmaz.

Adlandırma mantığı: “pull up” ve “pull down”, kalıtım hiyerarşisinde yukarı çıkarmak ve aşağı indirmek demektir. Bu mantıkla adlandırılmış başka refactoring eylemleri de olacak.

Hoca “kuzen sınıf” ile ne kastettiğini tahtada çizerek gösterdi: bir üst sınıfın iki alt sınıfı olsun (A ve B), kardeşlerinden birinin de iki alt sınıfı olsun (C ve D). Bu durumda A ve B sınıflarının kuzenleri C ve D sınıflarıdır.

Örnek (ekrandan, 00:40:00): `Salesman` ve `Engineer` sınıfları `Employee`’den türer ve ikisinde de aynı `getName` metodu vardır. Aslında en başta doğru yapılması gereken, `getName`’in üst sınıfta tanımlanmasıydı. Refactoring ile `getName` alt sınıflardan kesilip `Employee`’ye taşınır.

```
ÖNCE:                                SONRA:
        Employee                             Employee
           ^                                 getName
     ______|______                              ^
    |             |                       ______|______
 Salesman      Engineer                  |             |
 getName       getName                Salesman      Engineer
```

5.2 Özel durum: createBill ve chargeFor

İşler her zaman bu kadar basit olmaz (ekrandan, 00:42:30). Soyut bir `Customer` sınıfı var: `lastBillDate` (son fatura tarihi) üyesi ve yeni fatura eklemek için `addBill(dat: Date, amount: double)` metodu. Bundan iki alt sınıf türetilmiş: `Regular Customer` (normal müşteri) ve `Preferred Customer` (platin üye gibi ayrıcalıklı müşteri). İkisinde de `createBill(Date)` ve `chargeFor(start: Date, end: Date)` metotları var. `chargeFor`, başlangıç ve bitiş tarihleri arasındaki masrafın ne kadar faturalanacağını hesaplar ve iki sınıfta “çok farklı kodlanmış”tır. `createBill` ise iki sınıfta çok benzerdir.

Yapılan şey:

- `chargeFor`, hiç olmazsa üst sınıfa soyut olarak çekilir (slayttaki not: “chargeFor metodu üst sınıfta soyut tanımlandı”).
- `createBill(Date)` üst sınıfa çekilir; böylece içindeki yinelenen koddan kurtulunur. Farklılaşan kısım `chargeFor` olduğu için o alt sınıflarda farklı kalır.

```
SONRA:
Customer (soyut)
  lastBillDate
  addBill(dat: Date, amount: double)
  createBill(Date)
  chargeFor(start: Date, end: Date)   <- soyut
      ^
  ____|_____________________
 |                          |
Regular Customer         Preferred Customer
 chargeFor(start, end)    chargeFor(start, end)
```

Daha karmaşık durumlarda Template Method tasarım kalıbını kullanmak tavsiye edilir.

5.3 Template Method ile

Yinelenen kodun benzerliği birebir değilse, belli değişikliklerle kopyalanmışsa (örneğin 10 satırlık bir kod bloğunda ilk 3 ve son 3 satır tekrarlanıyor, aradaki 4 satırda ufak varyasyonlar var), en doğru çözüm tasarım kalıplarında görülen Template Method kalıbını kurmaktır. Tekrarlanan kısımlar üst sınıftaki template metotta yer alır; farklılaşan kısımlar alt sınıflarda gerçeklenen ve üst sınıfta soyut tanımlanan metotlara konur.

5.4 Kalıtım ilişkisi yoksa

Soru: Tekrarlanan ya da benzer kod birbiriyle kalıtım ilişkisi olmayan sınıflarda yer alıyorsa ne yapılır? Belki bu sınıflar parça-bütün ilişkisiyle bağlıdır, belki aralarında hiç ilişki yoktur.

Cevap: Böyle bir durum varsa tasarım büyük olasılıkla baştan yanlıştır; sınıflara sorumluluklar doğru atanmamıştır. Refactoring her şeyi baştan tasarlamak değildir. Belki baştan tasarlamak daha iyi olurdu, ama zaman yoksa, proje bitiş tarihi yaklaşmışsa ya da geriye uyumluluk çok önemliyse refactoring ile devam edilir. O zaman çıkarılan (extract edilen) metot, onu kullanan sınıflardan birine taşınır ve diğer sınıflar bu metodu o sınıftan çağırır. Belki yepyeni bir sınıf da oluşturulabilir; pek tercih edilmese de bu da kullanılabilir.

Özünde hep bir metot çıkarılıyor ve en uygun yerinin neresi olduğuna karar veriliyor. Kalıtım hiyerarşisinde yukarı taşınırsa bu Pull Up Method olur; Template Method kalıbı kurulursa o kalıp adını alır.

6. Decompose Conditional (Karar Verme Komutunu Parçalama)

Long Method sorununu çözmeye götüren üç eylem daha var: Decompose Conditional, Replace Temp with Query ve Replace Method with Method Object. Sonuncusu, ilk ikisi işe yaramadığında kullanılan daha karmaşık bir eylemdir.

6.1 Tanım

Bir `if` komutunun koşulu ya da bir `while` döngüsünün koşulu aşırı karmaşıksa bu koşullar daha küçük alt koşullara bölünür. “Compose” birleştirmek, “decompose” bunun tersi, bileşenlerine ayırmaktır. Aşırı karmaşık `if` komutlarını sadeleştirmek anlaşılabilirliği artırır, hata yapmayı engeller ve yapılan hatayı keşfetmeyi kolaylaştırır; koşul ne kadar karmaşıksa hata yapma olasılığı o kadar yüksektir.

6.2 Örnek

Kod (ekrandan, 00:50:00):

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;
```

`SUMMER_START` ve `SUMMER_END` sabitlerdir. Koşul şunu söylüyor: tarih yaz başlangıcından önce ya da yaz bitişinden sonra ise. Bunu anlamak gerçekten kolay değildir. Koşul doğruysa yapılacak işe bakarak anlamaya çalışılabilir: masraf, miktar çarpı kış oranına kış servis bedeli eklenerek bulunuyor; aksi halde yaz oranından faturalanıyor. Demek ki `if`'in içi “kışsa” demeye getiriyor. Ama `if` satırında yanlış kod olma olasılığı da vardır.

Hoca koşulun doğru olup olmadığını bir zaman çizgisi çizerek denetledi. Çizgi üzerinde yaz başlangıcı ve yaz bitişi işaretlenir. Bir şeyin sonu başlangıcından sonra olmalıdır; yıl döngüsel düşünülürse yaz başlangıcından önceki ve yaz bitişinden sonraki noktalar kıştır. Yani koşul doğrudur, ama uzun ve karmaşık yazılmıştır.

```
/*** SONRA: ***/
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);
private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
    date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
    return quantity * _summerRate;
}
private double winterCharge(int quantity) {
```

```
    return quantity * _winterRate + _winterServiceCharge;
}
```

Parametre olarak verilen tarihin yaz ayları içinde olup olmadığını belirleyen ayrı bir `notSummer` metodu yazılır. Bu metot private'tır; başka yerden çağrılmasının bir anlamı yoktur. Karmaşık koşul doğrudan bu metoda taşınmıştır. Hoca bunun için "çok da sadeleştirememişiz, biraz halının altına süpürmek gibi olmuş" dedi; ama en azından üstteki hesap mantığı anlaşılır hale geldi: tarih yaz değilse kışa göre, değilse yaza göre bedellendirme. `summerCharge` miktarı yaz oranıyla çarpar; `winterCharge` miktarı kış oranıyla çarpar ve kış servis bedelini ekler.

Kod hem kısaldı hem anlaşılır oldu. Yaz ve kış oranlarıyla ya da bir tarihin yaz içinde olup olmadığını anlamakla ilgili bir sorun varsa yalnız ilgili kısma odaklanmak yeterlidir. Üç yeni metot çıktı; metot çağırma ek yükü ihmal edilebilir; metotlar kendi içinde odaklanmış oldukları için anlaşılması daha kolaydır.

6.3 Düz mantıkla daha iyi bir sürüm

Bu, Fowler'ın kitabındaki birebir refactoring şeklidir. Hoca bunun daha da iyi yapılabileceğini söyledi: `notSummer` yerine `isSummer` metodu yazılıp `if (isSummer(date)) charge = summerCharge(quantity); else charge = winterCharge(quantity);` denebilir. Böylece karmaşık koşul da olumsuz yerine olumlu (düz mantıkla) yazılır. Hoca genelde ters mantığı daha zor anladığını, düz mantıktan gitmek daha rahat olacaksa öyle yapılması gerektiğini söyledi. Bu da hala bir refactoring'dir: dış arayüz değişmez, yalnız iç hesaplama daha anlaşılır şekilde farklılaşır. Hoca bu öneriyi slayda kendine not olarak ekledi.

> **Ek bilgi (derste anlatılmadı):** De Morgan kuralına göre `notSummer`'ın tersi `!date.before(SUMMER_START) && !date.after(SUMMER_END)` olur. Bu yazım, sınır günlerini (`SUMMER_START` ve `SUMMER_END` tarihlerinin kendisi) orijinal kodla aynı şekilde yaza sayar. `date.after(SUMMER_START) && date.before(SUMMER_END)` yazılırsa sınır günleri kışa düşer; davranış değiştiği için bu artık saf bir refactoring olmaz.

7. Long Parameter List (Uzun Parametre Listesi)

Aradan sonraki kod kusuru Long Parameter List'tir (ekrandan, 01:06:00). Slayttaki tanım:

- Bir metodun parametre sayısının fazla olması, hatta bu parametrelerden bazılarının başka metotlarda da birlikte geçmesi durumudur.
  - Demek ki aslında bu parametreler birlikte bir varlığın durum bilgisinin bir kısmını (1) veya tamamını (2) oluşturmaktadır.
- Çözüme götürecek düzenleme:
  - 1. Introduce Parameter Object: metot parametrelerinin sayısı çok ise azaltmaya yönelik bir düzenlemedir.
  - 2. Preserve Whole Object: metot parametrelerinin sayısı çok ise azaltmaya yönelik diğer bir düzenlemedir.

7.1 "Fazla" ne kadar?

Hoca bunun önemli bir soru olduğunu söyledi. Yazılım ölçütleri konusuna çok girmeden şunu belirtti: belirli bir kalitenin üstünde olduğu düşünülen yazılımlar çeşitli yazılım metrikleriyle incelenip olması gereken değer aralığı belirlenebilir. Bu aralığın ya da eşiğin üstünde ya da altında kalan değerler için "doğru değil" yargısına varılabilir, ama ayrıntılı incelemek gerekir. Diyelim bir metot beş parametre alıyor: acaba bu kadar parametre gerekli mi, ne yapılabilir?

7.2 Parametreler bir varlığı anlatıyor olabilir

Kodun bütününe bakınca çoğu zaman şu görülür: bu parametrelerin hepsi ya da çoğu başka metotlarda da birlikte geçer. Bir metot beş parametre alıyor, aynı beş parametreyi başka bir metot da alıyor, bunlardan dördünü alan bir başka metot daha var. Bu, birlikte geçen parametrelerin başlı başına anlamlı bir varlığın durum bilgisinin bir kısmını, belki tamamını oluşturduğunun sinyalidir.

Buradaki tasarım kusuru ilk haftalarda konuşulan ilkeyle ilgilidir: çözülmeye çalışılan problem hangi düzeyde parçalara ayrılıyorsa aynı düzeyde parçalara ayrılan tasarım iyi bir tasarımdır. Burada bu parçaların bir arada oluşturduğu bir bütün var, ama biz o parçaları ayrı parametrelerle ve sürekli aynı ya da benzer gruplamalarla aktarıyoruz.

8. Introduce Parameter Object (Parametre Nesnesi Oluşturma)

8.1 Tanım ve örnek

Bir metot çok fazla parametre alıyorsa uygun parametreler üye alan olarak yeni bir sınıfta toplanır ve o türden bir nesne parametre olarak kullanılır (ekrandan, 01:10:00).

```
ÖNCE:                                          SONRA:
Customer                                       Customer
  amountInvoicedIn(start: Date, end: Date)       amountInvoicedIn(DateRange)
  amountReceivedIn(start: Date, end: Date)       amountReceivedIn(DateRange)
  amountOverdueIn(start: Date, end: Date)        amountOverdueIn(DateRange)
```

Üç metodun hepsi parametre olarak bir başlangıç ve bitiş tarihi alır. İki parametre aslında uzun bir liste değildir; ama tekrarlandıkları görülünce bu ikisinin birlikte başlı başına bir varlık olup olmadığı düşünülür. Gerçekten öyledir: bunlar bir zaman aralığını, bir zaman dilimini ifade eder ve zaman dilimi bir başlangıç ve bitişle ifade edilir. Bu yüzden `start` ve `end` adlı iki `Date` üyesi olan `DateRange` sınıfı oluşturulur ve `Customer` sınıfının ilgili metotlarına artık bu türden bir nesne verilir.

8.2 Kazanç yalnız kısa parametre listesi değildir

Slayttaki maddeler (ekrandan, 01:10:00): bu iş yalnız parametre listesini kısaltmak amacıyla yapılmış olmaz; oluşturulan `DateRange` sınıfı yalnız iki veriyi birleştiren bir kayıt yapısı olarak kalmaz, `isWithinRange(Date)` gibi uygun metotları da içererek Decompose Conditional ya da Extract Method düzenlemesi de yapılmış olunur.

Hoca bunu bir önceki örnekle bağladı: tarihin yaz içinde olup olmadığı gibi denetimler kodun birçok yerinde büyük olasılıkla tekrar tekrar yapılıyordur. Bu kodu o farklı yerlerden çekip yeni `DateRange` sınıfına almak doğru bir karardır; kendisiyle ilgili denetimleri yapma görevi o sınıfın kendisine verilmelidir. Sonuçta yalnız iki veriyi birleştiren bir kayıt yapısı elde edilmedi; başka kod kusurlarının düzeltilmesine de vesile olan, uyumu yükselten bir refactoring eylemi yapıldı.

8.3 Arayüz değişiyor: bu hala refactoring mi?

Soru: `amountInvoicedIn` gibi metotlar önceden iki parametre alırken şimdi bir parametre alıyor; yani `Customer` sınıfının üç metodunun imzası değişti. Bu, refactoring'in temel ilkesini (dışarıya verilen hizmeti değiştirmeden iç çalışma biçimini değiştirmek) ihlal etmiyor mu?

Cevap: Hoca bu itiraza hak verdi: `Customer` sınıfı düzeyinde bakınca dışarıya verilen hizmetlerin arayüzü değişti. Ama sistemin geneline bakınca bu değişiklik sistemin geneli için yazılan testleri değiştirmez. Sınamalar düzey düzeydir: en küçük düzey birim testi, ardından tümleştirme (integration) testleri, ardından sistem testleri. Bu değişiklik sistem testlerinin kodunu (büyük olasılıkla tümleştirme testlerini de) değiştirmeyecektir; birim testlerini ise elbette değiştirecektir. Bu ufak bir sakıncadır (hoca bunun "kod kusuru" terimiyle karışmaması için "sakınca" dedi). Ama karşılığında elde edilen kazanımlar daha fazla olduğu için hoca bu refactoring eylemini bu şekilde yürütmeyi önerdi.

9. Preserve Whole Object (Tüm Nesneyi Gönder)

9.1 Tanım ve örnek

Bir nesnenin çeşitli üyelerini aynı metodun parametreleri olarak göndermek yerine nesnenin kendisi gönderilir (ekrandan, 01:20:00).

```
/*** ÖNCE: ***/
int low = daysTempRange().getLow();
int high = daysTempRange().getHigh();
withinPlan = plan.withinRange(low, high);

/*** SONRA: ***/
withinPlan = plan.withinRange(daysTempRange());
//withinRange metodu da gerektiği gibi değiştirilir.
```

Önceki kodda `low` ve `high` iki geçici değişkendir; biri `daysTempRange()` metodunun döndürdüğü nesnenin `getLow()` metodu, diğeri aynı nesnenin `getHigh()` metodu çağrılarak elde edilir. Sonra bu iki değer `plan` nesnesinin `withinRange` metoduna parametre olarak verilir. Bunun yerine, madem bu değerler `daysTempRange()`'in döndürdüğü nesnenin metotlarıyla elde ediliyor, nesnenin kendisi gönderilir; `withinRange` içinde ne hesap gerekiyorsa orada yapılır. Burada iki parametre var, ama 3, 4, 5 ya da daha fazla olabilir. Böylece Long Parameter List kusuru düzeltilebilir.

9.2 Birinci sakınca: bağımlılık

Slayttaki madde: bu düzenlemenin zayıf yönü bağımlılığın artabilmesidir; parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bir bağımlılık oluşur. Bu durum çok sakıncalı olacaksa bu düzenleme kullanılmaz.

Hoca örnek üzerinde düşündü: bu kodun yer aldığı sınıf ile `daysTempRange()`'in döndürdüğü nesne arasında zaten bir bağımlılık vardır, çünkü kod o türden bir nesneyi zaten alıyor. Yani o açıdan bağımlılık artmaz. Ama `withinRange` metodu daha önce iki tamsayı alırken şimdi bir nesne alıyor; demek ki `withinRange`'in yer aldığı sınıf ile bu nesnenin sınıfı arasında, UML sınıf şemasında kesikli çizgiyle gösterilen bir kullanma ilişkisi kurulmuş olur. Gereksiz yere bağlaşımı artırmak istemiyorsak bu eylemi yapmamayı tercih edebiliriz.

9.3 İkinci sakınca: aktarılan veri miktarı

Çağıran ile çağrılan metot arasında iki tamsayı (dörder bayttan sekiz bayt) aktarmak başka, bütün nesneyi aktarmak başkadır. Yerel bir uygulamada bu birkaç milisaniyelik ek yük getirebilir, ama genelde darboğaz başka yerdedir ve bu fark hissedilmez.

Bir adım ötesi düşünülürse: özellikle bankalar gibi finans kurumlarında çok farklı bilgi sistemleri olabilir ve iki bilgi sistemi arasında veri aktarımı zor, uzun ve masraflı olabilir. İki ayrı `int` yerine nesnenin bütün durum bilgisi gönderildiğinde, bu iki birim farklı ağlarda ya da farklı bilgisayarlarda yer alıyorsa arada ağ iletişimi olur. Belki daha masraflısı, bir GSM ağı üzerindeyse haberleşme masrafı artar. Böyle bir sakınca varsa eylem yapılmaz; yoksa yapılması tavsiye edilir.

Ayrıca yazılımın geliştirilmesi sürdükçe, şu an iki üye alana ihtiyaç duyulurken ileride bu sayı rahatlıkla 3-4-5'e çıkabilir ve parametre listesi uzar. Bu yüzden haberleşme süresi ya da masrafı anlamında önemli bir zarar yoksa bütün nesne gönderilmelidir.

10. Replace Temp with Query (Geçici Değişken Yerine Erişim Metodu)

10.1 Geçici değişkenlere neden "kıl" oluyoruz?

Tanım (ekrandan, 01:24:30): bir ifadeyi bir geçici değişkende saklayarak kullanmak yerine ifadeden metot(lar) çıkarılır.

Refactoring açısından geçici değişkenler pek tercih edilmez. Geçici değişkenler büyük olasılıkla çalışma anı başarımını iyileştirmek için kullanılır: hesap bir kez yapılır, sonucu birçok yerde kullanılır; aynı hesaba ihtiyaç olan yerlerde hesap boşuna tekrarlanmaz. Bu geçerli bir motivasyondur. Ama bakım aşamasına geçildiğinde (hataları düzeltirken, yeni gereksinim eklerken ya da mevcutları değiştirirken) bu karardan pişman olunabilir. Çünkü geçici değişken kullanımı kodun uzamasını ve karmaşıklaşmasını teşvik eder: "madem bu değişkeni hesapladım, şurada da kullanayım" derken metot gitgide uzar. Uzun metodun sakıncaları zaten tartı-

şılmıştı. Bu yüzden geçici değişkenle tutulan ifade tekrarlansın ya da yapılabiliyorsa Extract Method yapılsın.

10.2 getPrice örneği

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}

/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

(ekrandan, 01:24:30)

Önceki halde temel fiyat (`basePrice`) bir geçici değişkendir: miktar çarpı birim fiyat (`_quantity` ve `_itemPrice` alt çizgiyle başladığı için üye alandır). Sonra bir indirim oranı (`discountFactor`) geçici değişkeni tanımlanır: temel fiyat 1000'den büyükse `0.95` (%5 indirim), değilse `0.98` (%2 indirim). En sonda temel fiyat indirim oranıyla çarpılıp döndürülür.

`basePrice`'a yukarıda bir kez atama yapılmış, sonra hiç başka atama yapılmamıştır. `discountFactor`'a da ya `if`'te ya `else`'te bir kez atama yapılmıştır. Yani bu değişkenler yalnız hesap sonucunu saklamak için kullanılmıştır. Bu yüzden `getPrice` parçalanır: aslında yapılan şey, indirimsiz temel fiyatı hesaplamak, sonra indirim oranını belirlemek ve ikisini çarpıp döndürmektir. Beş satır tek satıra indi; komut sayısıyla bakılırsa dört komut yerine iki metot çağrısı oldu. "Çok da long method değildi ama yine de kısalttık, daha iyi oldu." Yeni `basePrice()` metodu üye alanları kullandığı için ek parametre aktarmaya gerek kalmaz.

10.3 Bir metot iki kez çağrılıyor

Hoca bir noktaya dikkat çekti: `getPrice()` çağrıldığında önce `basePrice()` hesaplanır, sonra `discountFactor()` çağrılır ve onun içinde `basePrice()` bir kez daha çağrılır. Yani önceki halde bir kez yapılan hesap şimdi iki kez yapılıyor. Basit, hafif bir metodu bir kez ya da iki kez çağırmak arasındaki başarım farkı hissedilemeyecek kadar küçüktür. Ama bir şerh var: finans kurumlarında olduğu gibi bu hesaplar ayrı bilgi sistemlerinden çekilecek sorgularsa aynı sorguyu iki kez yürütmek hem zaman kaybıdır hem de o sistemlerin sorgu yükünü artırır. Öyle durumlarda bu eylem uygulanmaz; yoksa uygulanır.

10.4 Avantaj, tartışma ve şerh

Slayttaki özet (ekrandan, 01:30:00):

- Avantaj: kopyala-yapıştır kodlamayı daha ortaya çıkmadan azaltmak.
  - Geçici değişkeni kullanan metodunkine benzer bir işlem eklenmek istendiğinde mevcut metottan yeni metoda kopyala-yapıştır yapılacak.
  - Bu kod parçacığı ile ilgili bir hata bulunursa tüm sınıfta buradan kopyala-yapıştır yapılan yerler aranıp her yerde düzeltme yapmak gerekecek.
  - Halbuki geçici değişkenlerle yapılan işlemler sorgu metotlarında toplanırsa düzeltmeler de sadece sorgu metotlarında yapılacaktır.
- Tartışma: bu düzenleme başarımı düşürür; sadece işlem yapmak yerine bir de fazladan metot çağırma yükü gelir.
- Şerh: modern derleyiciler bu ek yükü ortadan kaldırmış veya önemsizleştirmiştir.

Hoca, geçici değişkenin kopyala-yapıştırı "şeytanın dürtmesi" gibi teşvik ettiğini ve bunun sonunda en sorunlu kusurlardan biri olan Duplicated Code'un ortaya çıktığını söyledi. İşler kendi metotlarında kodlanırsa sorunun başı önceden kesilir: kopyalanacak her yerden zaten var olan metot çağrılır.

10.5 Döngüde çağrı sayısı: printRecipt

Slayttaki tartışma maddesi (ekrandan, 01:31:30): geçici değişken metot içinde birden fazla yerde kullanılıyorsa aynı işlem tekrar tekrar yapılacak; bu da başarımı daha da fazla düşürecek.

```
void printRecipt() { /*** ÖNCE: ***/
    int totalAmount = 0, thisAmount;
    for( Item item : items ) {
        thisAmount = item.getCharge( );
        System.out.println(item.getName()+" "+thisAmount);
        totalAmount += thisAmount;
} }

void printRecipt() { /*** SONRA: ***/
    int totalAmount = 0;
    for( Item item : items ) {
        System.out.println(item.getName()+" "+item.getCharge());
        totalAmount += item.getCharge( );
} }
```

Döngüde, ele alınan öğenin tutarı `item.getCharge()` ile alınır, öğenin adı ve tutarı yazılır ve tutar kümülatif toplama eklenir. İki geçici değişken vardır: `totalAmount` ve `thisAmount`. `totalAmount` için yapılacak bir şey yoktur; kümülatif toplamın bir birikimci değişkenine mutlaka ihtiyaç vardır. `thisAmount` ise bir kez `item.getCharge()` ile yazılır, sonra iki yerde yalnız okunur; başka hiçbir yerde ona atama yapılmaz. Bu yüzden `thisAmount` görülen her yere `item.getCharge()` yazılabilir.

Ama dikkatli bakınca `getCharge()` yalnız iki kez değil, öğe sayısının iki katı kadar çağrılır. Beş öğe varsa önceden 5 çağrı yapılırken şimdi $2 \times 5 = 10$ çağrı yapılır. Daha yüklü sayıda kayıt varsa bu ek yük hissedilebilir hale gelebilir (mutlaka gelir değil, gelme olasılığı var).

10.6 Profiler ile ölçmek

Ek yükün ihmal edilebilir olup olmadığı ölçülmelidir. Profiler adı verilen araçlarla hem test senaryolarının kaç milisaniye sürdüğü hem de bu sürede denetim akışının hangi satırlardan geçtiği ve hangi satırlarda ne kadar süre harcadığı ayrıntılı ölçülebilir. Refactoring'in öncesi ve sonrası profiler ile incelenir; ortaya çıkan başarım dezavantajı kabul edilebilecek düzeyin üstündeyse refactoring yapılmaz ya da yapılmışsa geri alınır; ihmal edilebilir düzeydeyse yapılması en doğrusudur.

11. Replace Method with Method Object (Metot Yerine Yeni Türden Nesne)

11.1 Tanım

Slayttaki maddeler (ekrandan, 01:36:50):

- Metot öyle uzun ve geçici yerel değişkenleri öyle "arap saçı" gibi kullanıyor ki Extract Method düzenlemesini kullanamıyorsunuz.
- O zaman yeni bir sınıf oluşturup bütün metodu o sınıfa taşıyın, yerel değişkenleri de o sınıfın üyeleri yapın.
- Ardından bildiğiniz düzenleme eylemlerini daha rahat yürütebilirsiniz.
- Yeni sınıfın adını verirken taşınan metodun adına benzer bir ad verin.

Hoca bunun bir avantajını da belirtti: sorun en azından izole edilir. İzole edilmiş yerinde belki daha kolay çözülür; çözülemese bile izole edilmiş olur.

11.2 Örnek

```
/*** ÖNCE: ***/
class Order {
   double price() {
   double primaryBasePrice;
   double secondaryBasePrice;
   double tertiaryBasePrice;
   // long computation;
   ...
}
```

```
SONRA:
Order                       PriceCalculator
  price()  - - - - - - ->     primaryBasePrice
     ^                        secondaryBasePrice
     |_______ 1 ____________ tertiaryBasePrice
                              compute()

price() içinde: return new PriceCalculator(this).compute()
```

(ekrandan, 01:36:50)

`Order` sınıfının `price()` metodunda üç geçici değişken (birincil, ikincil ve üçüncül temel fiyat) ve uzun bir hesap var. Bu metot komple, adı anlamlı olsun diye `PriceCalculator` adlı yeni bir sınıfa taşınır: madem metot fiyat hesaplıyor, taşındığı sınıfın adı da "fiyat hesaplayıcı" olsun. Metottaki bütün geçici değişkenler bu sınıfın üye alanları olur. Metodun adı artık `price` değil `compute` (hesapla) olur; `calculate` da denebilir.

`Order` sınıfında `price()` metodu hala yer alır, ama işi o yapmaz. Kendisini (`this`) vererek bir `PriceCalculator` oluşturur ve onun `compute()` metodunu çağırır: "benden aldığın bilgilerle ne hesap gerekiyorsa yap". `price()` küçük bir köprü, bir vekil (proxy) olur; esas işi kendisi yapmaz, yaptırır. Bu refactoring eylemi sistemin arayüzünü değiştirmez; `price()` hala aynı imzayla vardır.

12. Large Class (Aşırı Büyük Sınıf) ve Metrikler

12.1 Tanım

Uzun metotların çıkarabileceği sorunlara benzer şekilde, bir sınıfın kaynak kodu bütün olarak fazla büyük olabilir. "Bu büyüklük zaten uzun metotlardan gelir" denebilir, ama bu şart değildir: metotlar 3-7 satırlık küçük metotlar olabilir, ama çok fazla sayıda olabilirler. En temel neden genellikle sınıfın uyumunun düşük olması, yani sınıfa birbirinden farklı alanlardan sorumluluklar eklenmiş olmasıdır.

Slayttaki maddeler (ekrandan, 01:40:00):

- Sınıfın büyük olması, sınıfa ilgisiz sorumluluklar atandığının göstergesidir. Böylece uyum düşecektir.
- Üye alan sayısının fazlalığı, aşırı büyük sınıfın bir göstergesidir.
- Çözüme götürecek düzenlemeler:
  - Extract Class: birlikte anlam ifade eden üye alanlardan bir veya daha fazla yeni sınıf oluşturulur.
  - Extract Subclass: bazen oluşturulacak yeni sınıfın mevcut sınıfın bir alt sınıfı olması daha uygun olabilir. Örneğin mevcut sınıf bazı üyelerini her zaman kullanmıyorsa bunlar yeni bir alt sınıfa aktarılabilir.
  - Duplicate Observed Data: iş mantığı ile kullanıcı arayüzü arasında yüksek bağımlılık varsa GUI sınıfları çok büyüyecektir. Bu durumda veri yeni bir uygulama alanı sınıfına kopyalanır ve aradaki eşgüdüm Observer tasarım kalıbına göre sağlanır.

12.2 Ne kadar büyük "büyük"tür? Kaynak kod görselleştirme

Soru: Ne kadar büyük bir sınıf aşırı büyük sayılmalıdır?

Cevap: Hoca bunun da cevabı kolay olmayan bir soru olduğunu, en azından üzerinde çalışılan proje üzerinden daha rahat yanıtlanabileceğini söyledi. Bu iş için kaynak kod görselleştirme araçları (visualization tools) vardır. Hoca yazılım ölçümü konusunda güzel bir kitap olarak Lanza, Marinescu ve Ducasse'nin "Object-Oriented Metrics in Practice" kitabını gösterdi (ekrandan, 01:43:30); kitap üniversite kütüphanesinde basılı ve e-kitap olarak bulunuyor.

Kitaptaki "System Complexity" görünümünde (ekrandan, 01:43:30) her dörtgen bir sınıfı simgeler. Dörtgenler arasındaki bağlantılar kalıtım ya da kullanma ilişkisi olabilir. Daha önemlisi, dörtgenlerin boyutları ve rengi ayrı anlam taşır:

- Genişlik: sınıftaki üye alan (attribute) sayısıyla orantılıdır.
- Yükseklik: sınıftaki metot sayısıyla orantılıdır.
- Renk: renk koyulaştıkça sınıfta kod satırı (line of code) sayısı artar.

İnsan beyni böyle görsellerde en enli, en uzun, en koyu ya da en açık olanı bir bakışta hızla algılamakta çok yeteneklidir. Sistem genelinde refactoring yapılacaksa ve nereden başlanacağı düşünülüyorsa bu görselleştirmeler çok yararlıdır: örneğin en koyu, yani kod satırı en yüksek sınıftan başlanıp daha küçük parçalara ayrılmaya çalışılır. Diğerlerine göre daha enli bir sınıf görülürse üye alan sayısının gerekenden fazla olup olmadığı düşünülür.

Hoca görselde bazı enli sınıflara baktı ve bunları dengeli buldu, çünkü metot sayıları da ortalamadır. Çok metot olması kötü değildir, hatta iyi olabilir: tek uzun metot yerine daha kısa, daha çok metot oluşması zaten amaçlanır ve bu dörtgenleri uzatır. Kareye yakın görünen bir sınıfta ise sorumluluklar iyi dağıtılmamış olabilir; acaba uzun metotlar var mı diye sorgulamak yararlıdır.

12.3 Eşik değerler dile göre değişir

Fazlalık ölçütü kullanılan programlama yaklaşımına ve diline göre de değişir. Kitabın yazarları belirli kalite ölçütlerinin üstündeki 45 Java ve 37 C++ sistemini incelemiş, metriklerin ortalama değerlerini hesaplamış ve düşük, ortalama, yüksek ve çok yüksek gibi eşikler belirlemişlerdir; refactoring hedeflerinin buna göre belirlenmesini önermişlerdir. Ekrandaki tablo (ekrandan, 01:48:10; kitabın Tablo 2.2'si):

	Java				C++			
Metric	Low	Average	High	VeryHigh	Low	Average	High	VeryHigh
WMC	5	14	31	47	4	23	72	108
AMW	1.1	2.0	3.1	4.7	1.0	2.5	4.8	7.0
LOC/Class	28	70	130	195	20	90	240	360
NOM/Class	4	7	10	15	4	9	15	23

Hoca WMC'yi (Weighted Methods per Class) "sınıfın ağırlıklandırılmış metot sayısı" diye çevirdi ve bunların çok standart metrikler olduğunu, merak edenlerin araştırabileceğini söyledi; ayrıntıya girmedi. Ekrandaki sayfada WMC şöyle üç terime ayrılır: $WMC = \frac{CYCLO}{LOC} \cdot \frac{LOC}{Method} \cdot \frac{NOM}{Class}$.

> **Ek bilgi (derste anlatılmadı):** CYCLO, McCabe'nin döngüsel karmaşıklığıdır (bir metottaki bağımsız yol sayısı). AMW (Average Method Weight) metot başına ortalama döngüsel karmaşıklıktır (CYCLO/Method). LOC/Class sınıf başına ortalama kod satırı, NOM/Class (Number of Methods) sınıf başına ortalama metot sayısıdır. Tabloda C++ eşiklerinin çoğunlukla Java'dan yüksek olması, iki dilde sınıfların büyük olasılıkla farklı boyutlarda yazıldığını gösterir.

13. Extract Class (Sınıf Çıkartma)

13.1 Tanım

Nasıl Extract Method ile bir metodun içinden yeni metotlar çıkarılıyorsa, Extract Class ile de bir sınıfın içindeki bazı kısımlar alınıp yeni bir sınıf oluşturulur. Slayttaki maddeler (ekrandan, 01:52:30):

- Birbiri ile ilgisiz iki iş yapan bir sınıfı bu işlere göre ikiye ayırmaktır.
- İkinci iş ile ilgili üyelerin seçilerek yeni bir sınıfa taşınmasına dayanır.

- Bu yapılandırma da birçok kod kusurunu düzeltir.

Üç iş yapan bir sınıftan iki yeni sınıf çıkarılır, toplam üç sınıf olur.

13.2 Uyumla bağlantısı

Hoca bunu ilk haftalardaki uyum (cohesion) tartışmasına bağladı. Uyum ölçütleri genelde bir sınıfın üye alanlarına hangi üye metotlarla erişildiğini eşleştirir; bu eşleşmelerin sayısı tam bağlı (fully connected) eşleşme sayısından ne kadar küçükse uyum o kadar düşüktür. İlk hafta bu erişim grafiği uygun bir yerden kesilerek iki sınıf oluşturulmuş, iki sınıf kendi içinde tam bağlıya yakın çıkmış ve hem sınıfların bireysel uyumu hem sistem düzeyindeki uyum epey iyileşmişti. Extract Class ve Extract Subclass eylemleri de bunu yapar; elde edilen kazanım uyumun yükselmesidir.

13.3 Örnek: Person ve Telephone Number

```
ÖNCE:                    SONRA:
Person                   Person                        Telephone Number
  name                     name      officeTelephone     areaCode
  officeAreaCode           ------------------------>1    number
  officeNumber           getTelephoneNumber            getTelephoneNumber
getTelephoneNumber
```

(ekrandan, 01:52:30)

`Person` sınıfında isim bilgisi, ofis telefonunun alan kodu ve ofis telefon numarası var. Kişilerle birden fazla telefon ilişkilendirilecekse cep telefonu alan kodu, numarası gibi alanlar da eklenebilir ve sayı artar. Bir de telefonu bir string olarak döndüren `getTelephoneNumber` metodu var.

Burada bir tasarım sorunu vardır: problemde “kişinin telefonu” diye bir varlıktan söz edilebiliyorsa tasarım da telefon numarasını bölünmez, başlı başına bir bütün olarak içermelidir. Bu yüzden `officeAreaCode` ve `officeNumber` üyeleri alınıp yeni `Telephone Number` sınıfı (`areaCode`, `number`) oluşturulur ve bu üye alanlarla yapılabilecek işlemler esas olarak oraya kodlanır. `Person` sınıfındaki ilgili metot yalnız bir köprü (vekil) olur: yeni sınıftaki metodu çağırıp sonucu döndürür. Böylece iş tek bir üye alanla (`officeTelephone`) çözülür. Başka telefonlar eklenecekse her telefon için iki üye alan yerine kaç telefon eklenecekse o kadar yeni üye alan eklenir.

14. Extract Subclass (Alt Sınıf Çıkartma)

14.1 Tanım ve örnek

Bazen çıkarılan sınıfın mevcut sınıfın alt sınıfı olması daha mantıklıdır. Slayttaki tanım (ekrandan, 01:54:30): bir sınıfın bazı üyeleri her zaman kullanılmıyorsa bu üyelerden yeni bir alt sınıf oluşturmak daha doğru olacaktır.

```
ÖNCE:                SONRA:
Job Item             Job Item
  getTotalPrice        getTotalPrice
  getUnitPrice         getUnitPrice
  getEmployee             ^
                          |
                       Labor Item
                         getUnitPrice
                         getEmployee
```

(ekrandan, 01:54:30)

Sistemde bir `Job Item` sınıfı var: toplam fiyat, birim fiyat ve bu işten sorumlu çalışanı almak için `getEmployee` metodu. Hoca slayttaki diyagramın biraz rahat çizildiğini (üye alanların gösterilmediğini) ve kendisinin teknik diyagramlarda belirsizlikten uzak olmayı tercih ettiğini söyledi.

Bazı işler çalışana bağlı yapılır, bazıları çalışandan bağımsızdır. Bu fark edildiğinde aslında birbiriyle ilişkili olsa da iki ayrı varlıktan söz edildiği anlaşılır. Çalışana bağlı yapılan işler

daha özel bir durumu ifade eder. Buradan hemen kalıtım akla gelmelidir: kalıtımın anlamı alt sınıfların üst sınıfın daha özelleşmiş hali olmasıdır; üst sınıf alt sınıflara göre daha genel terimlerle ifade edilir. Bu yüzden kalıtıma genelleşme-özelleşme (generalization-specialization) ilişkisi de denir.

Çalışana bağlı işlerle ilgili kısımlar çıkarılarak `Labor Item` adlı yeni bir sınıf oluşturulur ve `Job Item`'ın alt sınıfı olarak sisteme eklenir. Her `Labor Item` bir `Job Item`'dır, ama her `Job Item` bir `Labor Item` değildir.

15. Duplicate Observed Data (Gözlenen Verinin Çoğullanması)

15.1 Sorun: iş mantığı GUI sınıfında

Hoca bu durumun bazen tembellikten kaynaklandığını söyledi. Programda iş mantığıyla ilgili çeşitli bilgiler kullanıcıdan alınmak zorundadır ve bu genelde formlarla yapılır. Ama bu formları aynı zamanda iş mantığı bilgilerinin saklandığı tek yer olarak kullanmak doğru değildir.

Slayttaki maddeler (ekrandan, 02:00:00):

- Uygulama alanı ile ilgili bir verinin sadece bir GUI sınıfında saklanması doğru değildir.
- Bu veri uygulama alanı/bilgi düzeyi katmanında oluşturulacak yeni bir sınıfa taşınmalı ve,
- GUI nesnesi ile yeni bilgi nesnesi arasında Observer tasarım kalıbı kurulmalıdır.

15.2 Örnek: Interval Window

```
ÖNCE:
Interval Window
  startField: TextField
  endField: TextField
  lengthField: TextField
  StartField_FocusLost
  EndField_FocusLost
  LengthField_FocusLost
  calculateLength
  calculateEnd

SONRA:
Interval Window  - - - - - - - - - - - - -|>  <<interface>> Observer
  startField, endField, lengthField: TextField
  StartField_FocusLost, EndField_FocusLost, LengthField_FocusLost
        |
        | 1
        v
Interval  ------------------------------|>  Observable
  start: String
  end: String
  length: String
  calculateLength
  calculateEnd
```

(ekrandan, 02:00:00)

`Interval Window` sınıfında bir şeyin başlangıcı ve bitişi girilir, sonra uzunluğu hesaplanır; belki buna göre ücretlendirme yapılır. `startField`, `endField` ve `lengthField` metin alanlarıdır. Bu sınıfta odak kaybı (`_FocusLost`) gibi GUI işlerine yönelik metotlarla birlikte `calculateLength`, `calculateEnd` gibi iş mantığına yönelik metotlar da yer alır. Bunlar kullanıcı arayüzü katmanındaki bir sınıfta olduğu için katmanlaşma bozulmuştur.

Aslında hesaplanması ve saklanması gereken şey bir aralıktır (interval); iç mantığı bu oluşturur. Bu iç mantığın parçası olan sınıftan nesne türetmek üzere bilgi toplanan pencere ise ayrı olmalıdır. Pencere yalnız GUI işlemlerine yönelik metotlar içerir (odak kaybedildi, küçültüldü, büyütüldü, düğmeye tıklandı gibi) ve iş mantığını yürüten `Interval` türünden bir nesneyle ilişkilidir.

İş mantığı nesnesinin neye göre ve nasıl güncelleneceği sorusuna cevap olarak Observer kalıbı kurulabilir. Observer kullanılmayacaksa belki Command kalıbı da kullanılabilir. Bu biraz ek iş getirir, ama yapılmazsa hem katmanlaşma bozulur hem de GUI sınıfı iç mantıkla birlikte aşırı şişer: hem long method hem large class olur. Ya en baştan böyle tasarlanmalı ya da bu yola kaçılmışsa düzeltilmelidir.

Örnek Sorular

1. (ekrandan, 00:30:00) Aşağıdaki `printOwing()` metodunda üç ayrı iş yorumlarla ayrılmıştır. Extract Method ile nasıl refactor edilir? Hangi metot dış arayüzde kalmalıdır?

```
void printOwing( ) {
  Enumeration e = _orders.elements();
  double outstanding = 0.0;
  // print banner
  System.out.println("*****************");
  System.out.println("* Customer Owes *");
  System.out.println("*****************");
  // calculate outstanding
  while (e.hasMoreElements()) {
    Order each = (Order) e.nextElement();
    outstanding += each.getAmount();
  }
  //print details
  System.out.println ("name:" + _name);
  System.out.println ("amount" + outstanding);
}
```

> **Çözüm (derste verildi):** Her iş kendi metoduna çıkarılır: `printBanner()`, `getOutstanding()` (for-each ile toplamı hesaplayıp döndürür) ve `printDetails(double outstanding)`. `printOwing()` dış arayüzde aynen kalır ve gövdesi `printBanner(); printDetails(getOutstanding());` olur. Çıkarılan metotlar başka yerden çağrılmayacaksa private olabilir. Yorum satırlarına artık gerek yoktur, çünkü metot adları işi anlatır.

2. (derste sözlü, 00:37:47) Yinelenen kod, birbiriyle kalıtım ilişkisi olmayan sınıflarda yer alıyorsa ne yapılır?

> **Çözüm (derste verildi):** Bu, tasarımın büyük olasılıkla baştan yanlış olduğunu, sorumlulukların doğru atanmadığını gösterir. Yeniden tasarıma zaman yoksa ya da geriye uyumluluk çok önemliyse refactoring ile devam edilir: kod bir metoda çıkarılır ve bu metot onu kullanan sınıflardan birine taşınır, diğerleri onu oradan çağırır; gerekirse yeni bir sınıf da oluşturulabilir. Sınıflar kalıtım hiyerarşisindeyse Pull Up Method, kopyalar arasında küçük farklılıklar varsa Template Method kalıbı kullanılır.

3. (ekrandan, 00:50:00) `if (date.before(SUMMER_START) || date.after(SUMMER_END)) charge = quantity * _winterRate + _winterServiceCharge; else charge = quantity * _summerRate;` kodunu Decompose Conditional ile refactor edin. Bu koşul gerçekten kışı mı anlatır?

> **Çözüm (derste verildi):** Koşul, yaz başlangıcından önceki ve yaz bitişinden sonraki tarihleri seçer; yıl döngüsel düşünülürse bunlar kıştır, yani koşul doğrudur. Refactor edilmiş hali: `if (notSummer(date)) charge = winterCharge(quantity); else charge = summerCharge(quantity);` ve üç private metot: `notSummer(Date)` koşulu döndürür, `summerCharge(int) quantity * _summerRate`, `winterCharge(int) quantity * _winterRate + _winterServiceCharge` döndürür. Hoca düz mantıkla `isSummer` metodu yazılıp `if (isSummer(date)) ... else ...` denmesinin daha da anlaşılır olacağını ekledi.

4. (derste sözlü, 01:12:03) Introduce Parameter Object ile `Customer` sınıfındaki `amountInvoicedIn(start, end)` gibi metotlar `amountInvoicedIn(DateRange)` yapıldığında metot imzaları değişir. Bu, "refactoring dış arayüzü değiştirmez" ilkesini çiğnemiş olmaz mı?

Çözüm (derste verildi): Sınıf düzeyinde çiğner; `Customer`'ın dışarıya verdiği hizmetlerin arayüzü değişir ve birim testleri değişmek zorunda kalır. Ama sistem düzeyinde tümleştirme ve sistem testleri değişmez. Ayrıca `DateRange` yalnız kayıt yapısı olarak kalmaz, `isWithinRange(Date)` gibi metotlarla başka kod kusurlarını da düzeltir. Kazanımlar sakıncadan büyük olduğu için hoca eylemin bu şekilde yapılmasını önerdi.

5. (ekrandan, 01:20:00) `int low = daysTempRange().getLow(); int high = daysTempRange().getHigh(); withinPlan = plan.withinRange(low, high);` kodu Preserve Whole Object ile nasıl yazılır? Bu düzenlemenin sakıncaları nelerdir?

Çözüm (derste verildi): `withinPlan = plan.withinRange(daysTempRange());` yazılır ve `withinRange` metodu buna göre değiştirilir. Sakıncalar: (1) bağımlılık artabilir; gönderilen nesnenin sınıfı ile `withinRange`'in sahibi olan sınıf arasında bir kullanma ilişkisi oluşur. (2) İki tamsayı yerine bütün nesne aktarılır; nesneler farklı bilgi sistemlerinde ya da ağ üzerindeyse süre ve haberleşme masrafı artabilir. Böyle bir sakınca yoksa eylem önerilir, çünkü ileride gereken üye sayısı artsa bile parametre listesi uzamaz.

6. (ekrandan, 01:24:30) `getPrice()` metodundaki `basePrice` ve `discountFactor` geçici değişkenlerini Replace Temp with Query ile kaldırın. Sonraki halde `getPrice()` bir kez çağrıldığında `basePrice()` kaç kez çağrılır?

Çözüm (derste verildi): `getPrice() { return basePrice() * discountFactor(); }`, `private int basePrice() { return _quantity * _itemPrice; }`, `private double discountFactor() { if (basePrice() > 1000) return 0.95; else return 0.98; }`. `basePrice()` iki kez çağrılır: biri `getPrice()` içinden, biri `discountFactor()` içinden. Basit bir metot için bu fark hissedilmez; ama hesap ayrı bir bilgi sistemine yapılan bir sorguysa eylem uygulanmamalıdır.

7. (ekrandan, 01:31:30) `printRecipt()` metodunda `thisAmount` yerine her yerde `item.getCharge()` yazılırsa ve listede 5 öğe varsa `getCharge()` çağrı sayısı nasıl değişir? `totalAmount` da kaldırılabilir mi?

Çözüm (derste verildi): Önceden her öğe için bir çağrı, toplam 5 çağrı yapılırken şimdi her öğe için iki çağrı, toplam $2 \times 5 = 10$ çağrı yapılır. Kayıt sayısı büyükse bu ek yük hissedilebilir; profiler ile öncesi ve sonrası ölçülüp karar verilir. `totalAmount` kaldırılamaz, çünkü kümülatif toplamın birikimci değişkenidir.

8. (derste sözlü, 01:39:33) Ne kadar büyük bir sınıf "aşırı büyük" sayılmalıdır?

Çözüm (derste verildi): Cevabı kolay değildir, en iyi üzerinde çalışılan proje üzerinden yanıtlanır. Kaynak kod görselleştirme araçları (örneğin genişliğin üye alan, yüksekliğin metot, rengin kod satırı sayısını gösterdiği "System Complexity" görünümü) aykırı sınıfları bir bakışta gösterir. Ayrıca belirli kalitenin üstündeki yazılımlardan türetilmiş metrik eşikleri kullanılabilir; bu eşikler dile göre değişir (örneğin Java için NOM/Class yüksek eşiği 10, C++ için 15; ekrandan, 01:48:10).

Dersin Toparlaması

Bu derste Fowler'ın "Refactoring" kitabından seçilen ilk dört kod kusuru ve onları düzelten refactoring eylemleri işlendi. Duplicated Code için Extract Method, Pull Up Method ve Template Method; Long Method için Extract Method, Decompose Conditional, Replace Temp with Query ve Replace Method with Method Object; Long Parameter List için Introduce Parameter Object ve Preserve Whole Object; Large Class için Extract Class, Extract Subclass ve Duplicate Observed Data görüldü. Her eylem "önce" ve "sonra" kodu ya da UML diyagramıyla gösterildi. Ortak fikir şuydu: anlaşılabilirliği ve uyumu artırmak, anlamlı adlarla yorumlara gerek bırakmamak, yinelenen kodu tek yerde toplamak ve sorumlulukları doğru sınıflara dağıtmak.

Hoca her eylemin bedelini de tartıştı: daha çok metot ve metot çağırma yükü, artan bağımlılık, aktarılan veri miktarı ve sınıf düzeyinde değişen arayüzler. Genel kural, kazanımın sakıncadan büyük olup olmadığına bakmak ve başarım kaygısı varsa profiler ile ölçmektir. Kaynak kod görselleştirme ve metrik eşikleri, refactoring'e nereden başlanacağına karar vermek için

önerildi. Kod kusurlarının kalan birkaç tanesi ve ardından video kiralama örneği üzerinde bütüncül refactoring sonraki derse kaldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.